

AIAA 3111 - 数据挖掘导论

第 5 讲

分类：基本概念

Dr. Zhidan Liu

香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

分类：两次课

第 1 次课：模型与假设	第 2 次课：训练与评估
监督学习与数据划分	线性回归与逻辑回归
决策树：划分与过拟合	评估指标、留出法与交叉验证
贝叶斯、朴素贝叶斯与 kNN	Bagging 、 Boosting 与随机森林
模型比较	类别不平衡

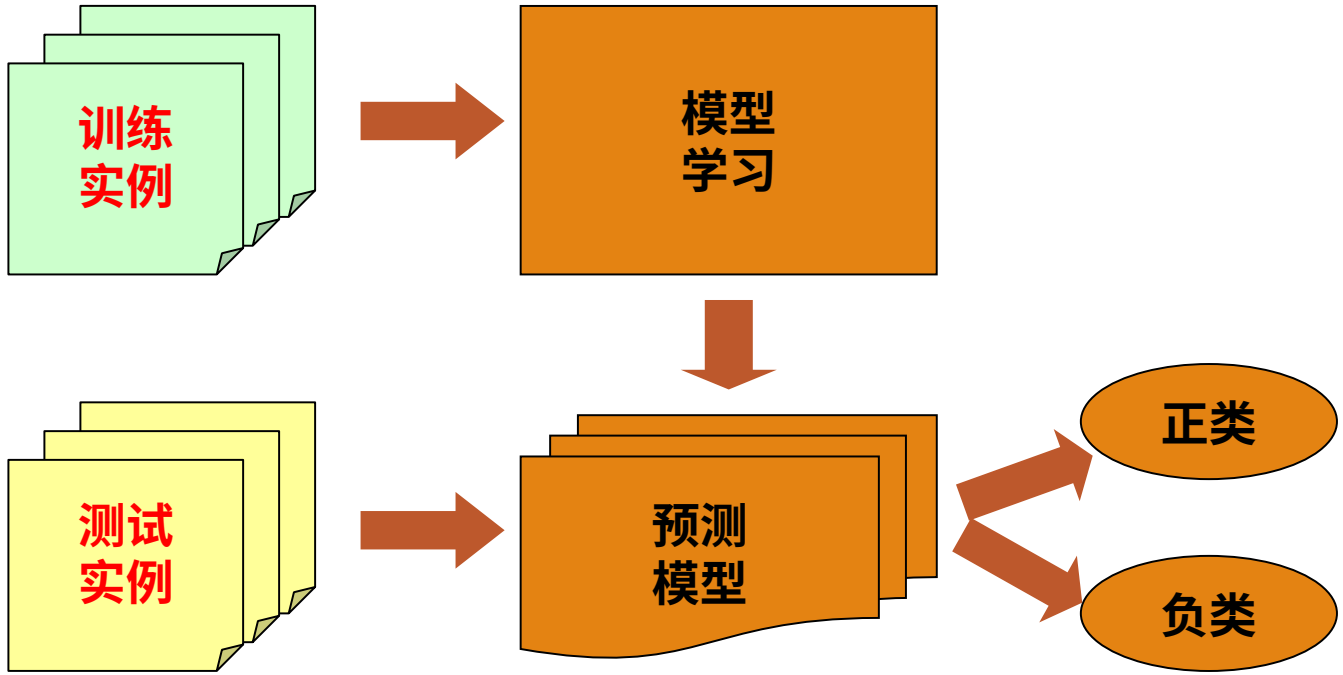
遵循同一流程：定义目标 → 训练 → 验证 → 评估。

监督学习

- 监督学习（分类）
  - 监督：训练数据（如观测或测量值）附带**标签**，指明其所属类别
  - 新数据依据由训练集构建的模型进行分类

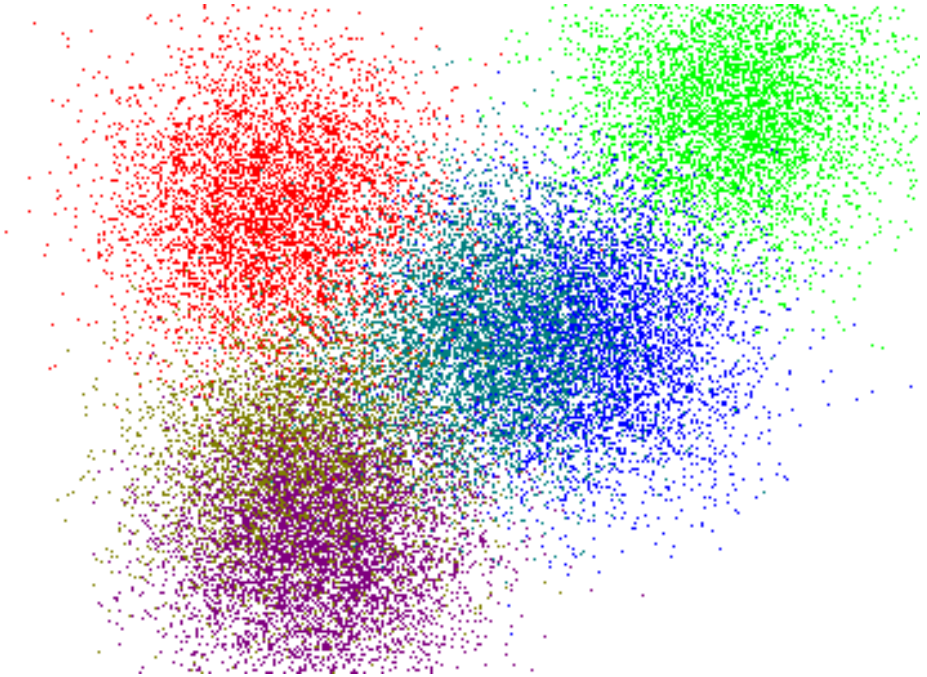
带**类别标签**的训练数据：

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No



无监督学习

- 无监督学习 （聚类）
  - 训练数据的类别标签**未知**
  - 给定一组观测或测量值，确定数据中可能存在的类别或簇





半监督学习

- 训练时结合少量有标签数据与大量无标签数据。
- 目标仍是监督预测；无标签样本提供额外的结构信息。

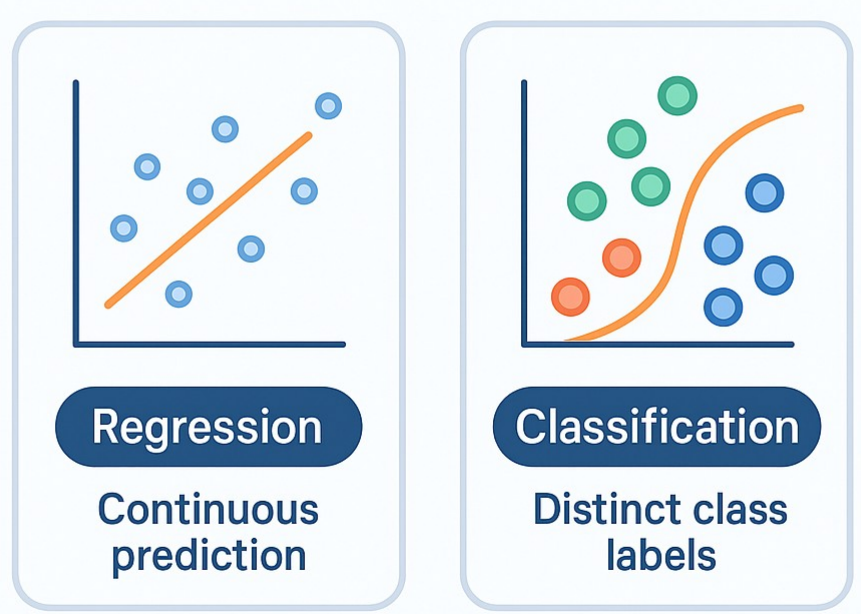
数据来源	对学习的贡献
有标签样本	定义目标类别并提供误差信号
无标签样本	揭示输入分布，支持更平滑的决策边界

- **自训练**把高置信度预测作为临时标签。
- 一致性方法或基于图的方法鼓励相似样本得到相容的预测。

应在真实有标签的验证数据上评估；无标签数据也可能强化模型错误。

分类 vs. 回归

- **分类**
  - 预测**类别型标签**（离散或名义）
  - 基于训练集及其**类别标签**（*分类属性的取值*）构建模型，并用它对新数据分类
- **数值预测**
  - 对**连续值函数**建模（即预测未知或缺失的值）



训练、验证与测试

- **训练数据**：拟合模型参数或学习决策树。
- **验证数据**：选择设置，如树深度或 kNN 中的 k。
- **测试数据**：在选择确定后估计性能。

阶段	可以学习什么？
训练	树的划分、类别概率、回归权重
验证	超参数、模型选择、决策阈值
测试	在未见样本上的最终性能估计

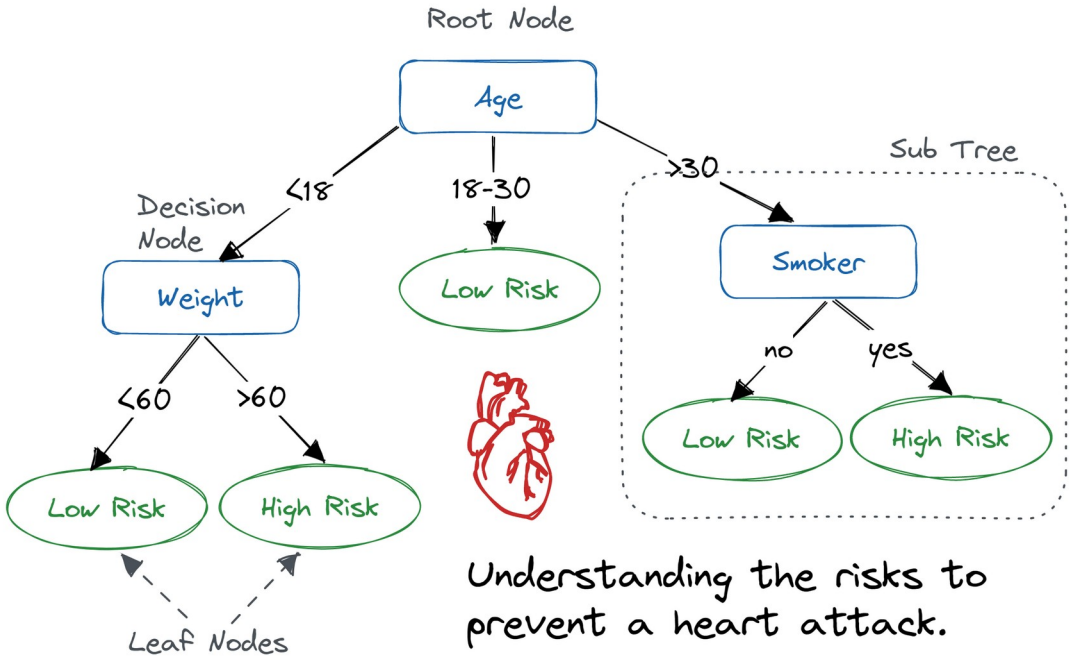
无数据泄漏的预处理

- 先划分数据，再学习预处理步骤。
- 插补、缩放与特征选择只在训练集上拟合。
- 将拟合好的变换应用于验证与测试数据。
- 必要时将相关记录放在一起：同一病人或用户不得跨划分泄漏。

例： kNN 需要缩放；只从训练数据学习均值与尺度。

决策树：从提问到预测

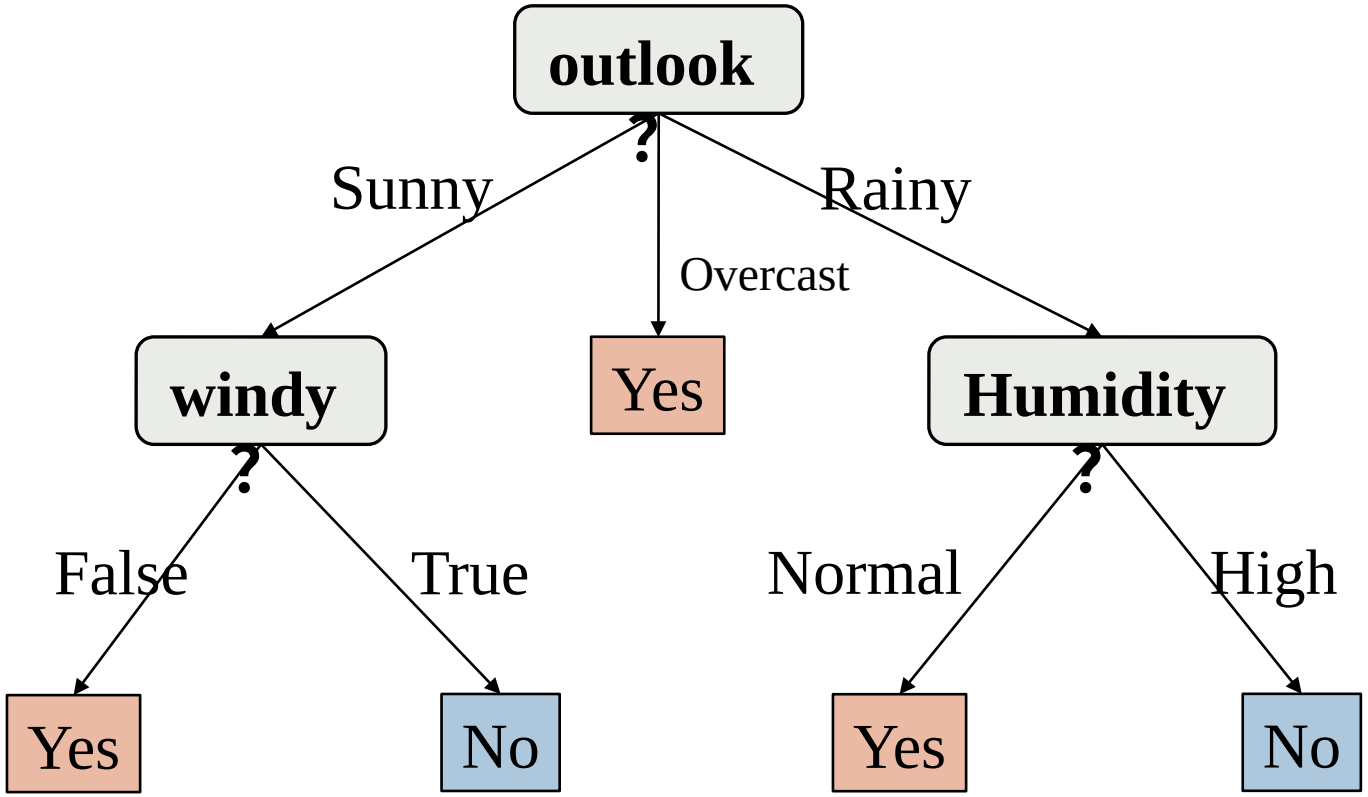
- 内部结点检验一个属性；一条边对应一个检验结果。
- 叶结点预测一个类别，通常为其训练样本中的多数类。
- 训练时选择使子结点类别更**纯**的划分。
- **预测沿从根到叶的一条路径进行。**



关键问题：选哪个划分？树何时停止生长？

决策树归纳：示例

- **决策树构建：**
  - 自顶向下、递归、分而治之的过程
- **得到的树：**



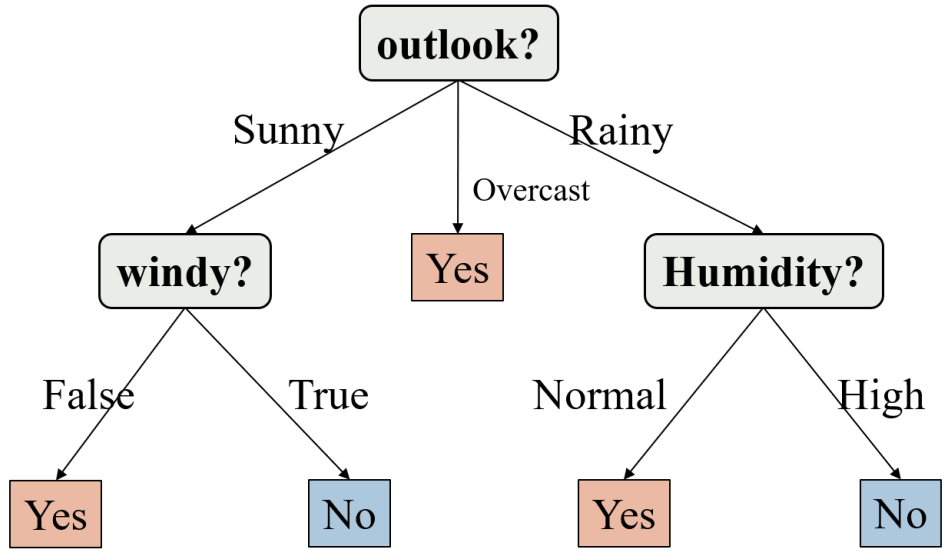
训练数据集：是否打高尔夫?

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

https://www.saedsayad.com/decision_tree.htm

决策树归纳：算法

- **基本算法**
  - 树以**自顶向下、递归、分而治之的方式**构建
  - 开始时，所有训练样本都在根结点
  - 根据选定属性递归地划分样本
  - 在每个结点上，依据该结点上的训练样本以及某种启发式或统计度量（如**信息增益、基尼指数**）选择属性



决策树：何时停止

- 结点上所有样本同属一类 → **建立叶结点。**
- 没有有效划分 → **预测该结点的多数类。**
- 没有训练样本到达某分支 → **使用父结点的多数类。**
- 达到深度或最小叶大小限制 → **停止以控制复杂度。**

一个连续属性可在多个结点被选中，对每个数据子集使用不同的划分阈值。

```
Age ≤ 40?
├─ Yes → Age ≤ 25?
└─ No  → Age ≤ 60?
```

划分连续值属性

- 方法 1：将连续值离散化，当作类别值处理
  - 例如 age： < 20 、 [20,30) 、 [30,40) 、 [40,50) 、 ≥ 50
- 方法 2：为连续值属性 A 确定***最佳划分点***
  - 排序：如 15, 18, 21, 22, 24, 25, 29, 31, …
  - *候选划分点：* $(a_i+a_{i+1})/2$
    - e.g., (15+18)/2 = 16.5, 19.5, 21.5, 23, 24.5, 27, 30, …
  - 选择*不纯度下降最大*的阈值（如信息增益）
- 划分：依据划分点 P
  - 为连续属性 A 选择划分点 P，并把 D 划分为：
    - $D_{\text{left}} = \{x \in D \mid A(x) \leq P\}$
    - $D_{\text{right}} = \{x \in D \mid A(x) > P\}$

度量不纯度：熵

- **好的划分会降低类别标签的不确定性**。
- 对结点 D，令 p_j 为属于类 j 的***比例***。

$$H(D) = -\sum_j p_j \log_2(p_j)$$

二分类结点	熵（比特）	解释
10 个 Yes， 0 个 No	0	纯
5 个 Yes， 5 个 No	1	最不确定
9 个 Yes， 5 个 No	≈ 0.940	打高尔夫训练集

选择划分：信息增益

- 一次划分把 D 分成子结点 $D_1, \ldots, D_v$。
- 计算划分后的*加权*平均不纯度。
- **选择不纯度下降最大的划分。**

$$\text{Gain}(D, A) = H(D) - \sum_v (|D_v| / |\mathbf{D}|) H(D_v)$$

不同决策树使用不同的度量：

ID3： 信息增益。 C4.5： 增益率。 CART： 通常为基尼下降。

打高尔夫： Outlook 的信息增益

Outlook	Yes	No	合计	熵
Rainy	2	3	5	≈ 0.971
Overcast	4	0	4	0
Sunny	3	2	5	≈ 0.971
全部记录	9	5	14	≈ 0.940

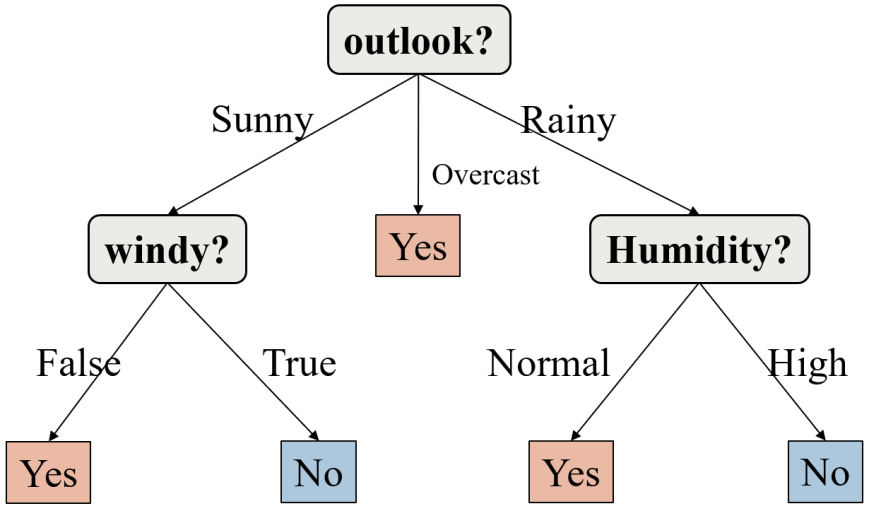
H(after) = (5/14)(0.971) + (4/14)(0) + (5/14)(0.971)

信息增益 = 0.940 – 0.694 ≈ 0.246 比特。

打高尔夫：比较候选属性

- 各增益都基于同一父结点计算： H(D) ≈ 0.940 。

属性	子结点加权熵	信息增益
Outlook	≈ 0.694	≈ **0.246**
Temperature	≈ 0.911	≈ 0.029
Humidity	≈ 0.788	≈ 0.151
Windy	≈ 0.892	≈ 0.048



在根结点选择 Outlook；然后在每个子结点内重复。

增益率：修正多路划分

- 信息增益可能偏好取值很多的属性（如 ID ）。
- 增益率用**分支大小的熵**对增益做归一化。

$$\mathrm{SplitInfo}(A) = -\sum_v q_v \log_2(q_v), \quad q_v = |D_v| \,/\, |D|$$

$$\mathrm{GainRatio}(A) = \mathrm{Gain}(A) \,/\, \mathrm{SplitInfo}(A)$$

Temperature： 分支大小 4, 6, 4 → SplitInfo ≈ 1.557

GainRatio ≈ 0.029 / 1.557 ≈ 0.019

C4.5 还会避免选择信息增益很小的划分。

另一种划分准则：基尼不纯度

$$Gini(D) = 1 - \sum_j p_j^2$$

结点	计算	Gini
10 个 Yes， 0 个 No	$1 - 1^2 - 0^2$	0
5 个 Yes， 5 个 No	$1 - 0.5^2 - 0.5^2$	0.50
9 个 Yes， 5 个 No	$1 - (9/14)^2 - (5/14)^2$	≈ 0.459

- 用子结点的加权基尼值为划分打分。
- 子结点不纯度越低，划分越好。

选择决策树划分

所有候选划分都遵循同一不纯度下降流程。

子结点加权不纯度	$I(children) = \Sigma_v (\lvert D_v \rvert / \lvert D \rvert) I(D_v)$
不纯度下降	$\Delta I = I(D) - I(children)$

通用流程

1	计算父结点不纯度
2	计算各子结点不纯度
3	按子结点大小加权
4	选择下降最大者

准则与划分得分

准则	候选划分得分	最优
信息增益	H(D) − H(children)	**最大**
增益率	信息增益 / 分裂信息	**最大**
基尼下降	Gini(D) − Gini(children)	**最大**

不同准则公式不同，但遵循相同的划分选择流程。

快速检查：哪个划分更好？

问题：父结点： 5 个 Yes 、 5 个 No 。比较两个候选划分，哪个产生*更纯的子结点*？无需计算。

候选	左子结点	右子结点
划分 A	3 个 Yes ， 3 个 No	2 个 Yes ， 2 个 No
划分 B	4 个 Yes ， 1 个 No	1 个 Yes ， 4 个 No

A. 划分 A　　B. 划分 B　　C. 不纯度相同

答案： B 。

控制树的复杂度

方法	原理	示例
预剪枝	提前停止生长	最大深度；最小叶大小
后剪枝	先生长，再简化	移除对验证几乎无益的分支

- 更深的树通常更好地拟合训练数据，但可能过拟合。
- 用验证数据或交叉验证选择复杂度。
- 未动用过的测试集只用于最终评估。

用验证数据选择树深度（模型复杂度）或剪枝强度。**测试集只用于最终评估。**

ID3、C4.5 与 CART

三种广泛使用的决策树。

三者都贪心地生长树。区别在于划分准则、划分形式和复杂度控制。

ID3

- 准则：信息增益
- 通常为多路划分
- 主要用于类别属性
- 原始方法不剪枝
- 分类

C4.5

- 准则：增益率
- 多路或阈值划分
- 类别 + 连续
- 缺失值处理 + 后剪枝
- 分类

CART

- 基尼或平方误差下降
- 仅二叉划分
- 类别 + 连续
- 代价复杂度剪枝
- 分类 + 回归

C4.5 扩展了 ID3； CART 是独立的二叉树框架，也支持回归。

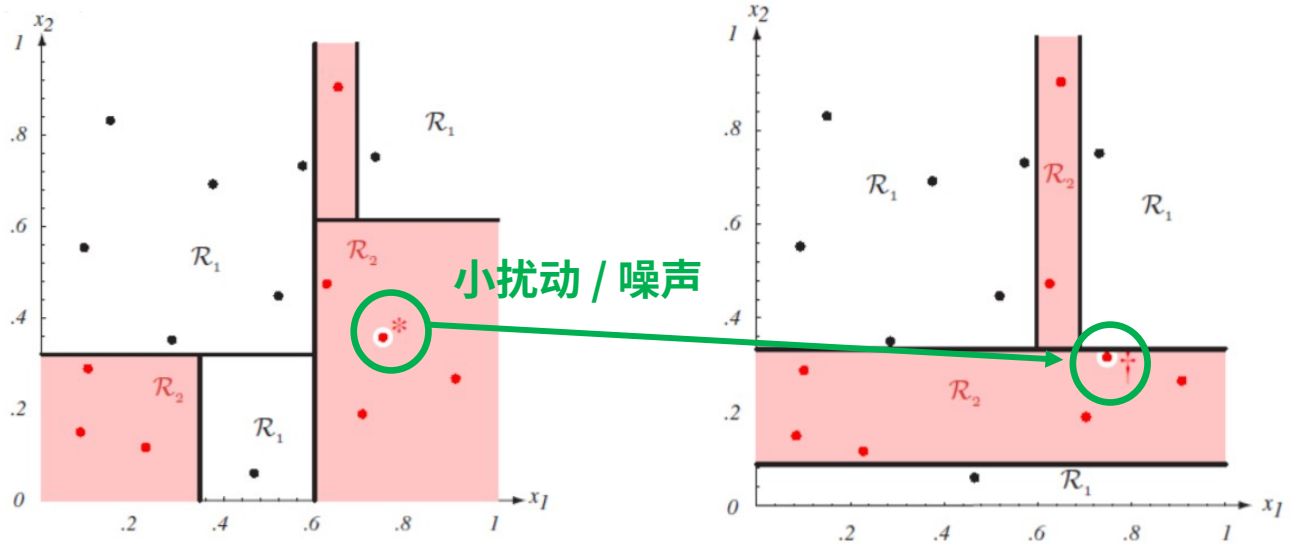
决策树：优点

- **优点**
  - 易于解释（即使对非专业人士）
  - 易于实现（有众多软件）
  - 预测快速：只需沿一条从根到叶的路径
  - 缺失值处理取决于具体算法
  - 小树的规则可解释
  - 无需对数据归一化
  - 非参数：不假设固定分布或特征独立
  - 可处理多种属性类型

决策树：局限

- 局限
  - 不稳定，对噪声敏感
  - 寻找全局最优树很困难；实用算法采用*贪心划分*
    - 贪心划分不能保证得到全局最优树。
  - 过拟合问题

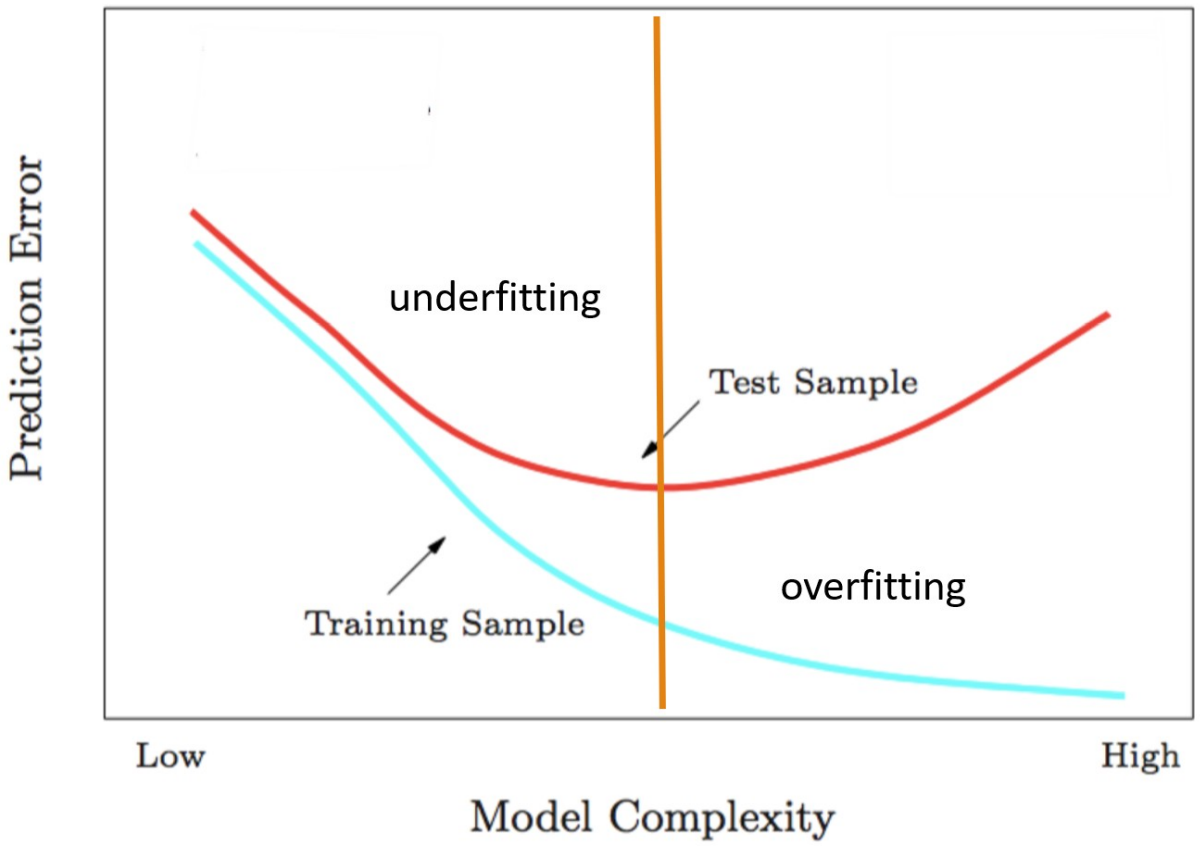
R1 / R2： 预测的决策区域



树 A： 一种贪心划分序列　　树 B： 小扰动导致不同的划分

训练拟合相近 ≠ 泛化相同

模型复杂度：欠拟合与过拟合



- **低复杂度**：两种误差都高
  树过于简单。
- **最佳复杂度**：验证误差最小
  树的泛化最好。
- **高复杂度**：训练误差下降但验证误差上升
  树过拟合了训练数据。

决策树与贝叶斯分类

两者都预测类别标签，但组织证据的方式不同。

决策树

- 学习划分特征空间的检验。
- 一条路径确定包含 x 的叶结点。
- 叶结点预测标签或类别比例。

问题： x 落在哪里?

贝叶斯分类器

- 从先验 $P(C)$ 与似然 $P(x \mid C)$ 出发。
- 贝叶斯定理计算后验 $P(C \mid x)$ 。
- 预测选择后验最大的类别。

问题：哪个类别最能解释 x ?

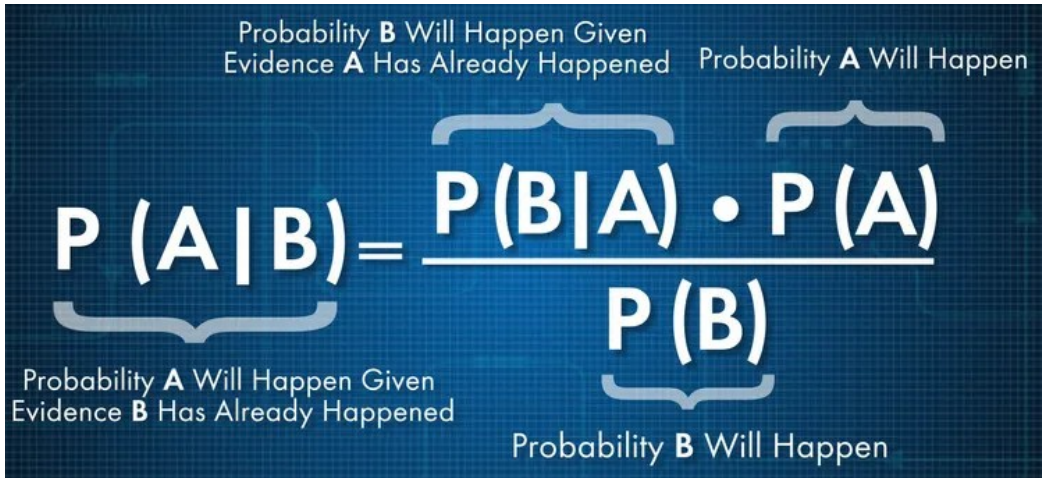
$$P(C \mid x) = P(x \mid C)\, P(C) / P(x)$$

决策树通过划分表示证据；贝叶斯模型通过概率表示证据。

贝叶斯定理：反转条件

$$P(C \mid x) = P(x \mid C)\, P(C) / P(x)$$

术语	在分类中的含义
P(C)： 先验	在看到 x 之前，类 C 有多常见?
P(x | C)： 似然	若类别为 C， x 出现的可能性有多大?
P(C | x)： 后验	观察到 x 之后， C 的可能性有多大?



P(C | x) 与 P(x | C) 回答的是不同问题。

贝叶斯定理示例：野餐日

- 早晨多云。
- 下雨的概率是多少?
- 所有雨天中有 50% 早晨是多云的。
- 多云的早晨很常见（40% 的日子早晨多云）
- 本例假设 P(Rain) = 0.10 。

P(Rain | Cloud) = ?

P(Cloud | Rain) = 50%

P(Cloud) = 40%

P(Rain) = 10%

$$P(C \mid x) = P(x \mid C)\, P(C) / P(x)$$

P(Rain | Cloud) = P(Rain) P(Cloud | Rain) / P(Cloud) = 10% * 50% / 40% = 12.5%

- 下雨的概率是 12.5%，而不是 50% 。
- 贝叶斯定理连接了后验与似然。
- P(Rain | Cloud) 与 P(Cloud | Rain) 的分母不同。

朴素贝叶斯：一种简单的概率分类器

- 用特征 $x = (x_1, \ldots, x_d)$ 表示一个样本。
- **假设在给定类 C 时各特征条件独立。**

$$P(x \mid C) \approx \prod_j P(x_j \mid C)$$

$$\text{score}(C) = P(C) \prod_j P(x_j \mid C)$$

预测得分最大的类别；*对得分归一化*即得概率。

朴素贝叶斯：为每个特征的似然建模

- 类别型特征：由各类别内的计数估计概率。

$$P(x_j = v \mid C) = \text{count}(x_j = v, C) / \text{count}(C)$$

- 连续型特征：高斯朴素贝叶斯使用概率密度。
- **为每个类别中的每个特征估计均值 μ 和标准差 σ。**

$$f(x \mid C) = [1 / (\sigma\sqrt{(2\pi)})] \exp[-(x - \mu)^2 / (2\sigma^2)]$$

将各特征似然与类别先验相乘，为每个类别打分。

打高尔夫：一个特征，两个后验

- 在 5 条 Sunny 记录中，3 条为 Yes，2 条为 No。

类别： play golf= ‘yes’ play golf = ‘no’

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

打高尔夫：训练数据集

量	由计数得到
P(Yes | Sunny)	3 / 5 = 0.60
P(No | Sunny)	2 / 5 = 0.40
P(Sunny | Yes)	3 / 9 ≈ 0.333

反转条件时，分母会改变。

朴素贝叶斯分类器示例：似然表

- 训练数据集共有 14 个样本：
  - 9 个“yes”样本
  - 5 个“no”样本

Frequency Table

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3

Likelihood Table

		Play Golf	
		Yes	No
Outlook	Sunny	3/9	2/5
	Overcast	4/9	0/5
	Rainy	2/9	3/5

		Play Golf	
		Yes	No
Humidity	High	3/9	4/5
	Normal	6/9	1/5

		Play Golf	
		Yes	No
Temp.	Hot	2/9	2/5
	Mild	4/9	2/5
	Cool	3/9	1/5

		Play Golf	
		Yes	No
Windy	False	6/9	2/5
	True	3/9	3/5

朴素贝叶斯：对一条新记录分类

x = (Rainy, Cool, High, True)

$$P(x \mid C) \approx \prod_j P(x_j \mid C)$$

$$\text{score}(C) = P(C) \prod_j P(x_j \mid C)$$

类别	先验 × 特征似然	得分
Yes	(9/14) × (2/9) × (3/9) × (3/9) × (3/9)	0.00529
No	(5/14) × (3/5) × (1/5) × (4/5) × (3/5)	0.02057

P(Yes | x) ≈ 0.205　　P(No | x) ≈ 0.795　　**（归一化后）**

预测： No 。将每个得分除以两个得分之和。

避免零概率：拉普拉斯平滑

- 只要有一个特征似然为零，整个类别得分就为零。
- 对 K 个取值，每个计数加 1，类别总数加 K。

$$P(x_j = v \mid C) = [count(v, C) + 1] / [count(C) + K]$$

Income （类 C 内）	观测计数	平滑后概率
Low	0	1 / 1003
Medium	990	991 / 1003
High	10	11 / 1003

朴素贝叶斯：优点与局限

优点	局限
快速训练与预测	*条件独立假设可能不现实*
参数少；数据有限时有用	相关特征可能重复计算证据
文本分类的强基线	分布的选择很重要（如高斯）
简单的按类概率估计	概率估计可能校准不佳

选择与数据相符的特征似然模型，并验证结果。

急切学习 vs. 惰性学习

	急切学习
查询之前	学习显式的预测模型
预测时	应用学到的模型
示例	决策树；朴素贝叶斯
主要开销	通常集中在训练阶段

惰性学习仍需要距离度量、预处理以及 k 的选择。

惰性学习器：基于实例的方法

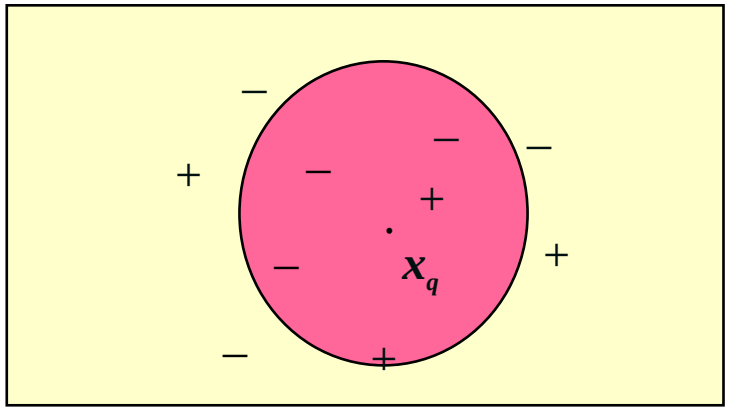
- 基于实例的学习：
  - 存储训练样本，推迟处理（“惰性求值”），直到必须对新实例分类
- 典型方法
  - k 近邻方法
    - 实例表示为欧氏空间中的点。
  - 局部加权回归
    - 构建局部近似
  - 基于案例的推理
    - 使用符号表示与基于知识的推理

k 近邻算法

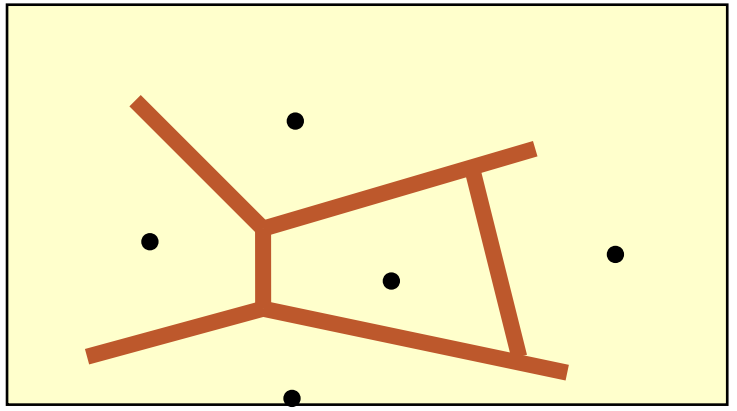
- 所有实例对应 n 维空间中的点
  - 假设每个实例用 n 个特征表示
- 常用的距离度量是欧氏距离
- 目标函数可以是离散值或实数值

- 对离散值， k-NN 返回与 x_q *最近*的 k 个训练样本中最常见的值

- **Voronoi 图**： 1-NN 在一组典型训练样本上诱导的决策面



(a) 单个查询的局部投票



(b) 1-NN 诱导的全局划分

关于 k-NN 算法的讨论

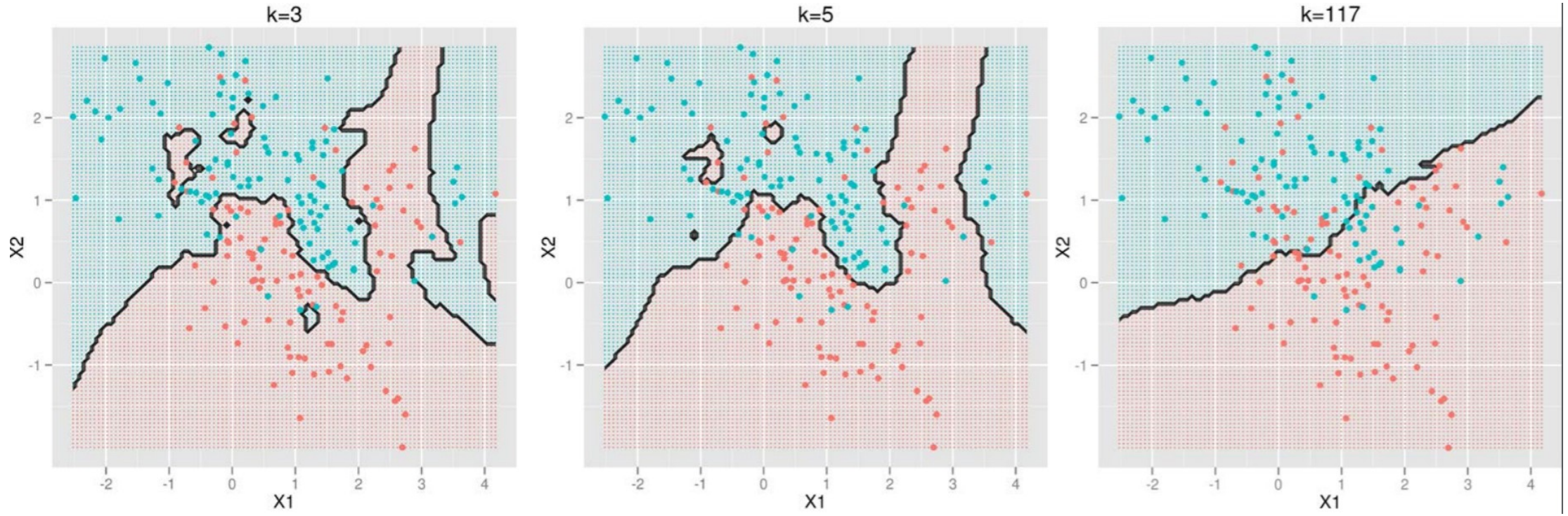
- k-NN 用于对给定未知元组做实值预测
  - 返回 k 个最近邻的均值
- 距离加权最近邻算法
  - 按 k 个近邻与查询 x_q 的距离对其贡献加权
    - 距离越近的近邻权重越大

$$w = \frac{1}{d(x_q, x_i)^2}$$

- 对近邻取平均 / 投票可降低对个别噪声样本的敏感性
- 维度灾难：近邻间的距离可能被无关属性主导
  - *缩放特征并考虑特征选择*；验证这些选择

kNN 中 k 的选择

- **近邻数 k**
  - k 小：过拟合（**高方差、低偏差**）
  - k 大：边界过度平滑，欠拟合（**高偏差、低方差**）



随着 k 增大，决策边界变得更平滑。用验证数据或交叉验证选择 k。

http://scott.fortmann-roe.com/docs/BiasVariance.html

快速检查：诊断流程

问题： kNN 使用年龄（岁）和收入（美元），收入主导了欧氏距离。该怎么办？

A	增大 k 直到训练准确率达 100%
B	用训练数据缩放特征，再验证 k
C	在划出测试集之前缩放整个数据集
D	用测试集选择要删除的特征

答案： B。 **请同时解释距离问题与数据泄漏问题。**

第 1 次课：模型、假设与复杂度

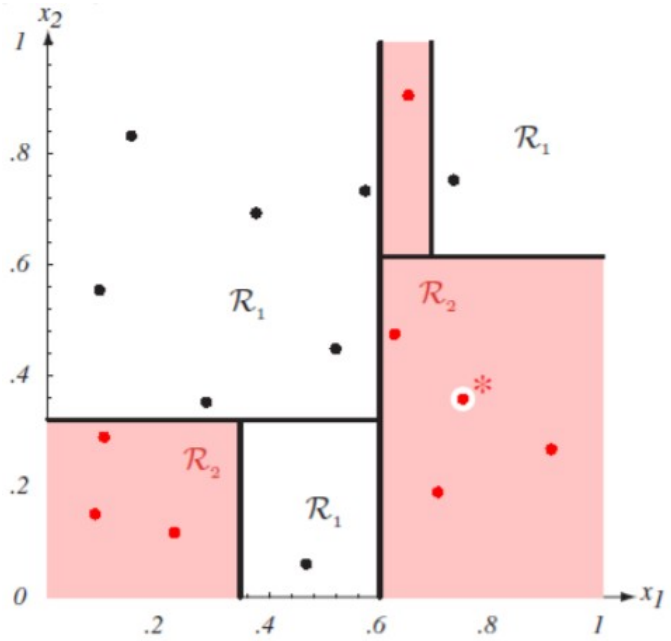
模型	核心思想	需要注意什么？
决策树	递归地进行降低不纯度的划分	深度、叶大小与过拟合
朴素贝叶斯	先验 × 按类似然	独立性、平滑、似然模型
kNN	由邻近样本预测	缩放、距离、 k 与维度

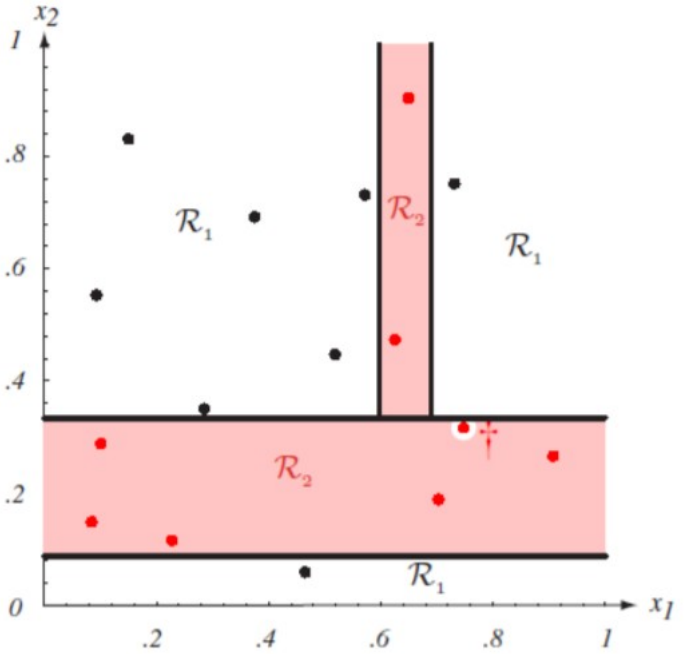
- 训练数据用于拟合，验证用于选择，测试用于报告。
- **训练拟合好并不保证在未见数据上表现好。**

第 1 次课附录

决策树：局限

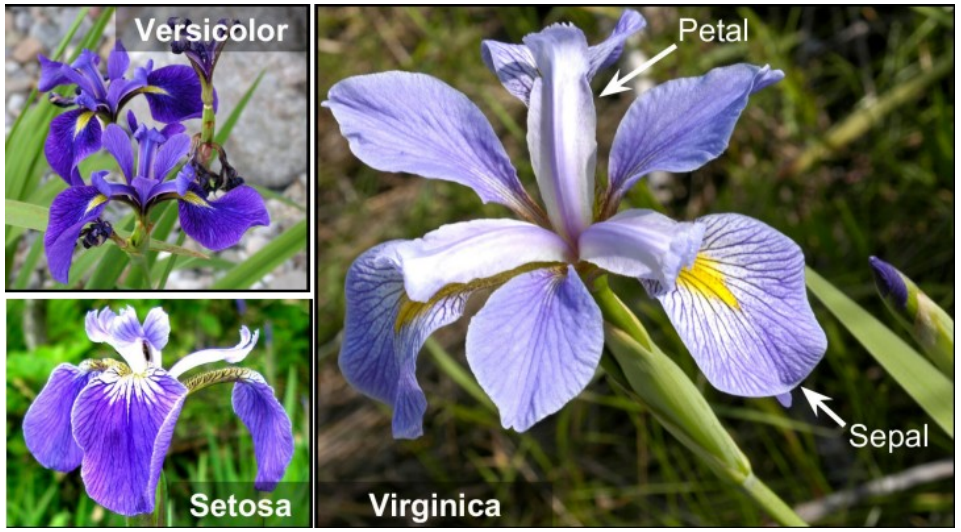
- 局限
  - 不稳定，对噪声敏感
  - （较低的）准确率可能不够好（取决于数据）
  - 寻找全局最优树很困难；实用算法采用*贪心划分*
  - 过拟合





附录 1： 鸢尾花分类练习

- 自己动手试一个例子！
  - https://scikit-learn.org/stable/modules/tree.html



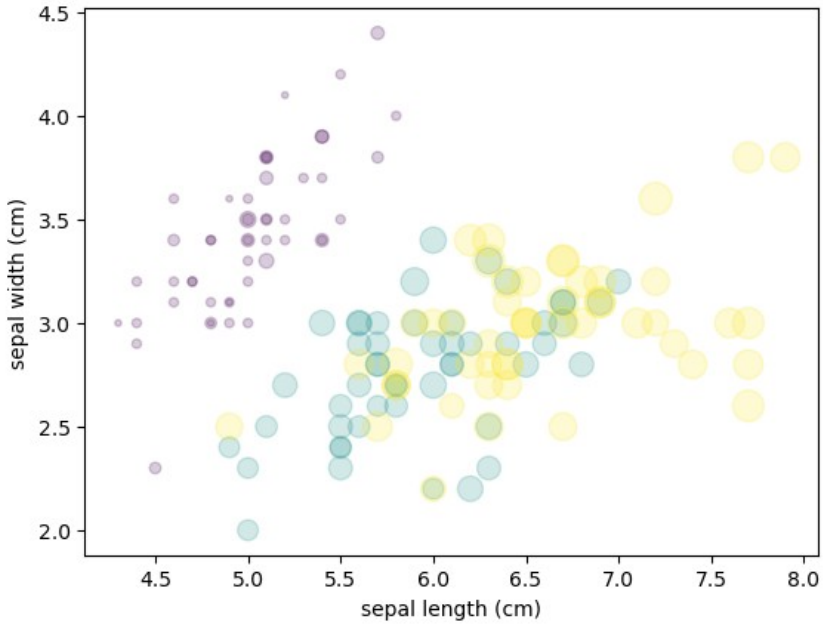
著名的 Iris （鸢尾花）数据集

```
from sklearn.datasets import load_iris

iris = load_iris()
features = iris.data.T

plt.scatter(features[0], features[1], alpha=0.2,
            s=100*features[3], c=iris.target, cmap='viridis')

plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1]);
```



附录 1：基于案例的推理

- **CBR**：利用问题解决方案数据库来解决新问题
- 案例可使用超越简单特征向量的结构化描述
- 应用：客户服务（产品相关诊断）、法律裁决
- 方法
  - 实例用丰富的符号描述表示（如函数图）
  - 搜索相似案例，可组合多个检索到的案例
  - 案例检索、基于知识的推理与问题求解紧密耦合
- 挑战
  - 找到好的相似度度量
  - 基于句法相似度建立索引；失败时回溯，并适应更多案例

附录 1：一个小型 kNN 练习

查询 q 有以下近邻（由近到远）。

近邻	到 q 的距离	类别
1	1.0	A
2	1.0	A
3	2.0	B
4	3.0	B
5	4.0	B

- 分别用 k = 3 和 k = 5 预测。
- k = 3 时，用权重（1/ 距离）进行预测。

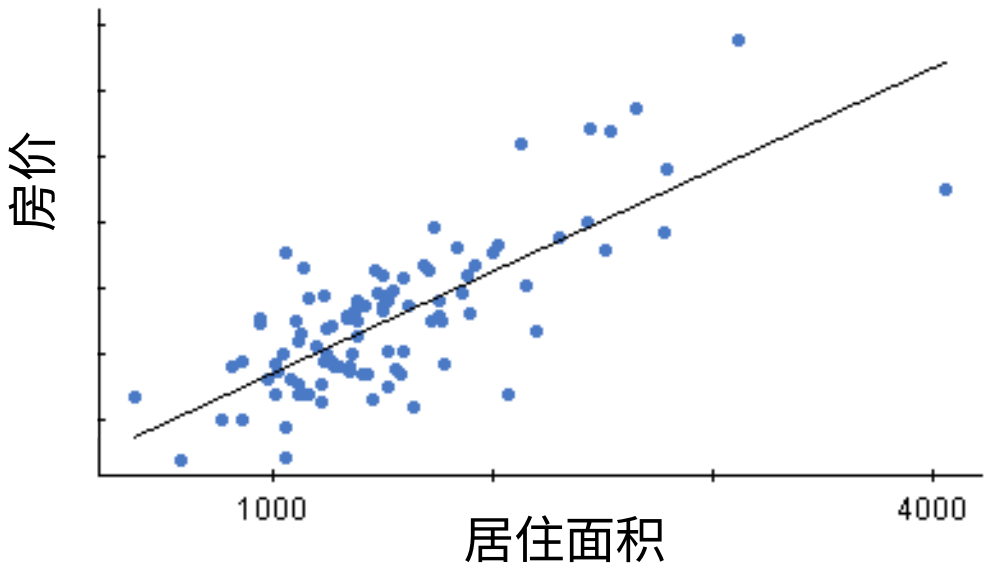
第 2 次课：训练、评估与改进

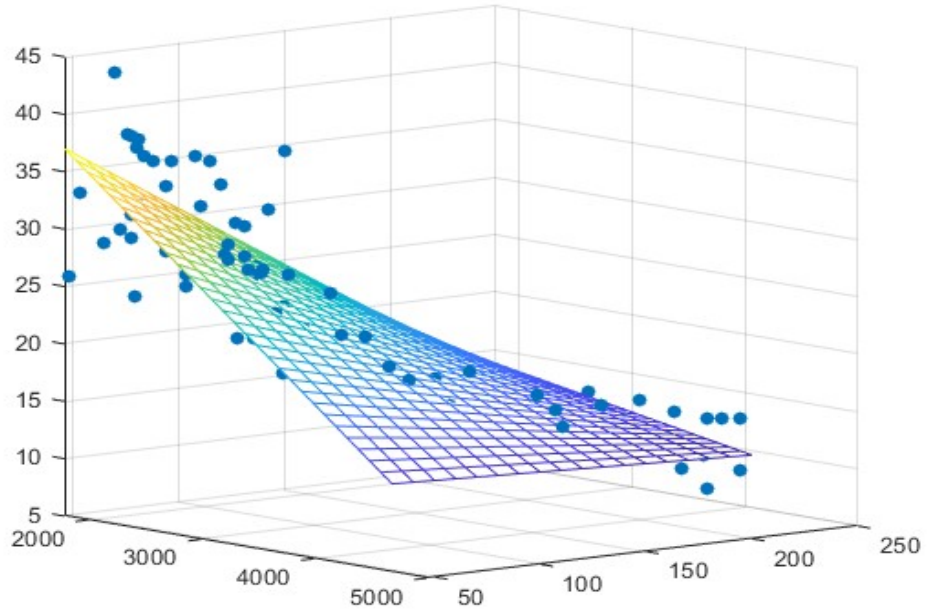
主题	主要问题
线性 / 逻辑模型	如何学习得分与概率?
评估	如何衡量泛化能力?
集成	多个模型如何起作用?
不平衡 / 回顾	对任务而言哪些错误重要?

让预测任务、数据划分与评估指标保持一致。

线性回归问题：示例

- 从自变量属性映射到**连续值**： x => y
- { 居住面积 } => 房价
- { 大学；专业； GPA} => 未来收入





线性回归：模型与参数

- 训练数据： n 个样本 (x_i, y_i)， x_i 含 d 个特征。
- 预测值是加权和加上偏置项。

$$\hat{y}_i = \mathbf{w}^T\mathbf{x}_i + \mathbf{b} = w_1x_{i1} + \cdots + w_dx_{id} + b$$

参数	作用
$w_1, \ldots, w_d$	其他特征不变时，该特征每增加一个单位预测值的变化
b	所有特征为零时的预测值（即偏置）

线性回归：最小二乘拟合

- 残差 = 观测值 − 预测值。
- 选择 w 和 b 以*最小化*均方误差。

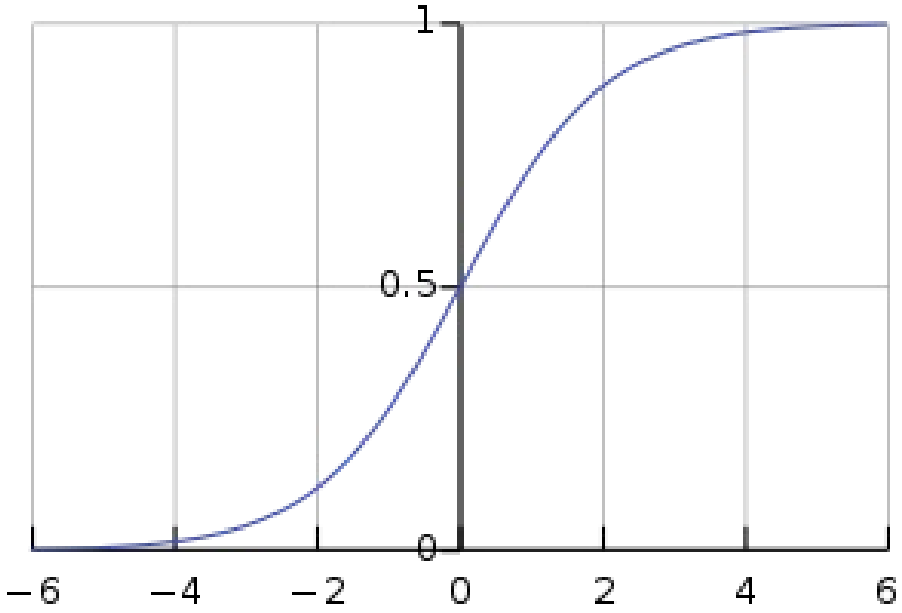
$$MSE(w, b) = (1/n) \sum_i [y_i - (w^T x_i + b)]^2$$

观测 y	预测 ŷ	残差	残差平方
3	2	1	1
5	7	−2	4

对这两个样本： MSE = (1 + 4) / 2 = 2.5 。

逻辑回归： Sigmoid 函数

- 线性得分 z 可以是任意实数。
- *Sigmoid* 把 z 映射到 0 到 1 之间。
- 得分越大，预测为类 1 的概率越高。

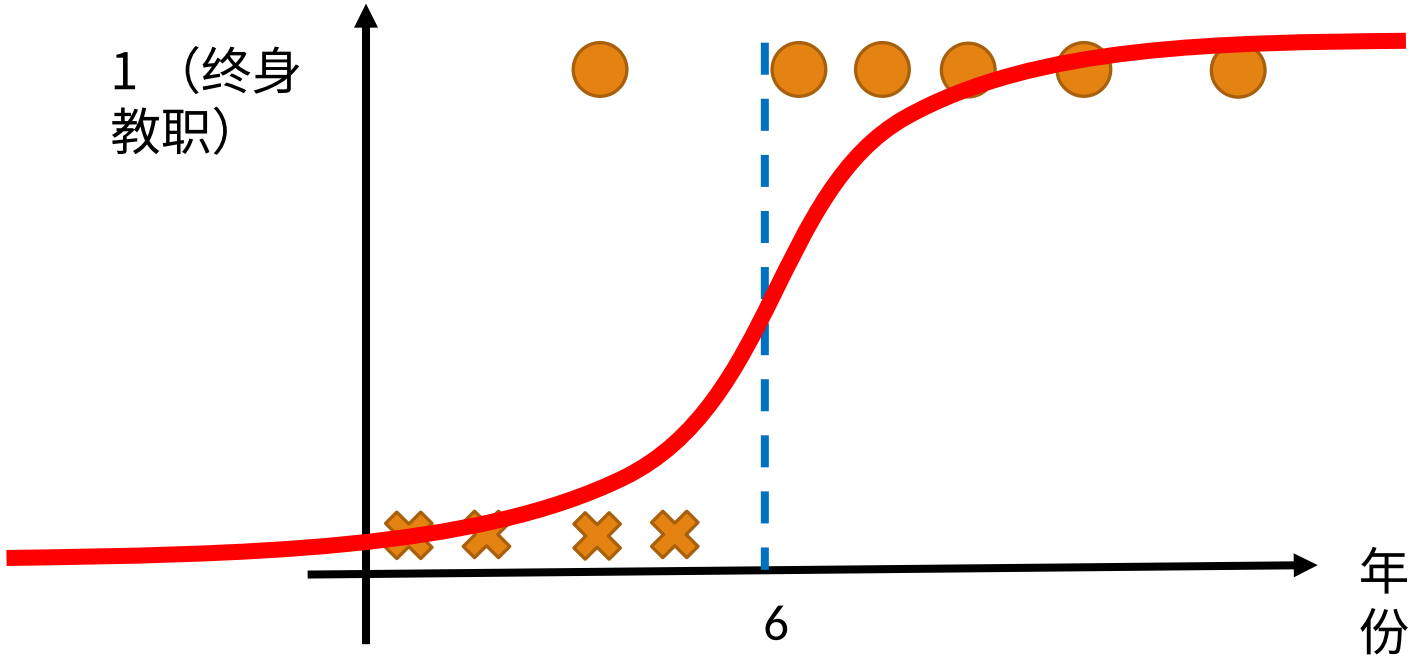


$$\sigma(z) = 1 / (1 + e^{-z})$$

σ(0) = 0.5；线性得分决定在曲线上的位置。

逻辑回归：示例

- 假设只把年份作为特征
  - 对线性得分应用 sigmoid；二元标签保持不变



逻辑回归：从得分到概率

$$z = w^T x + b \qquad p = P(Y = 1 \mid x) = 1 / (1 + e^{-z})$$

线性得分 z	概率 p	阈值为 0.5 时
−2	≈ 0.119	预测 0
0	0.500	决策边界
2	≈ 0.881	预测 1

阈值为 0.5 时，边界为 $w^T x + b = 0$ 。

学习逻辑回归：对数损失

- 对标签 $y = 1$，好的模型给出高概率 p。
- 对标签 $y = 0$，好的模型给出低概率 p。

$$L = -(1/n) \sum_i [y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)]$$

真实标签	预测 p	该样本的损失
1	0.9	$-\ln(0.9) \approx 0.105$
1	0.1	$-\ln(0.1) \approx 2.303$

自信却错误的预测会受到很大惩罚。

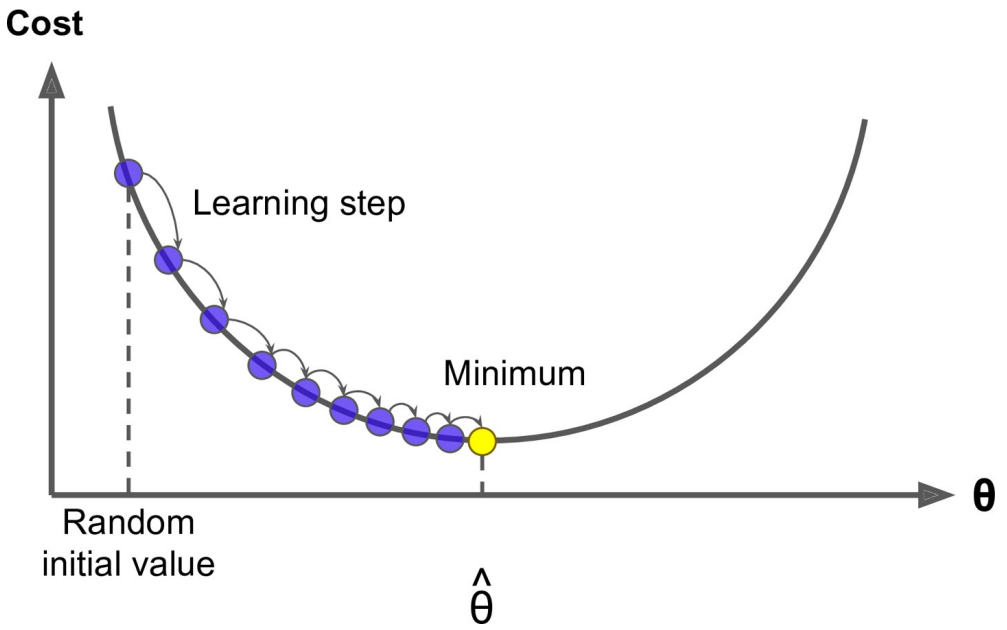
梯度下降

- Gradient Descent is *an iterative optimization algorithm for finding the minimum of a function* (e.g., the negative log likelihood)
- 沿梯度反方向走一小步可降低目标函数

步长

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \gamma \nabla F(\mathbf{a}_n)$$

梯度为零 ≠ 一定是最小值



逻辑回归：训练循环

- 初始化权重和偏置。
- 计算得分 z_i 与概率 p_i。
- 计算损失梯度；更新参数。
- **重复**直到满足停止规则；验证模型设置。

$$w \leftarrow w - \eta \, (1/n) \sum_i (p_i - y_i) x_i$$

$$b \leftarrow b - \eta \, (1/n) \sum_i (p_i - y_i)$$

η 为学习率。*特征缩放通常有助于优化。*

概率、预测与阈值

- 选定阈值 t 后，概率估计才变成类别预测。
- 若 p ≥ t 预测为类 1，否则预测为类 0。
- 降低 t 通常会提高*召回率*，同时增加*假正例*。
- **依据任务的错误代价或约束，在验证数据上选择 t。**

例： p = 0.35 → t = 0.50 时为类 0，但 t = 0.20 时为类 1。

评估：三个不同的问题

问题	工具
我们犯了哪些错误?	混淆矩阵
哪种性能重要?	精确率、召回率、 F 值； ROC / PR 分析
能否泛化到新数据?	留出测试；验证 / 交叉验证

- 明确指出正类。
- 在相同划分、相同指标下比较模型。

分类器评估指标：混淆矩阵

- **混淆矩阵：**

实际类别 \ 预测类别	C_1	$\neg C_1$
C_1	**真正例（TP）**	**假负例（FN）**
$\neg C_1$	**假正例（FP）**	**真负例（TN）**

- 在含 m 个类别的混淆矩阵中，$CM_{i,j}$ 表示类 i 中被分类器标为类 j 的元组数
  - 可增加行 / 列给出合计
- **混淆矩阵示例：**

实际类别 \ 预测类别	play_golf = yes	play_golf = no	合计
play_golf = yes	**6954**	**46**	7000
play_golf = no	**412**	**2588**	3000
合计	7366	2634	10000

准确率、召回率与特异度

A\P	C	¬C	
C	TP	FN	P
¬C	FP	TN	N
	P’	N’	All

真实情况

预测

- **分类器准确率**，或识别率
  - 测试集中被正确分类的元组所占百分比
    - Accuracy = (TP + TN)/All
- **错误率**： *1 – 准确率*，或
  - Error rate = (FP + FN)/All

- **类别不平衡问题**
  - 某个类别可能很*稀少*
    - 例如欺诈、 HIV 阳性
  - 负类占*绝大多数*，正类占少数
  - 除准确率外，还要报告各类别的性能
    - **灵敏度**（召回率）：真正例识别率
      - Sensitivity = TP/P
    - **特异度**：真负例识别率
      - Specificity = TN/N

精确率与召回率

A\P	C	¬C	
C	**TP**	**FN**	**P**
¬C	**FP**	**TN**	**N**
	P'	**N'**	**All**

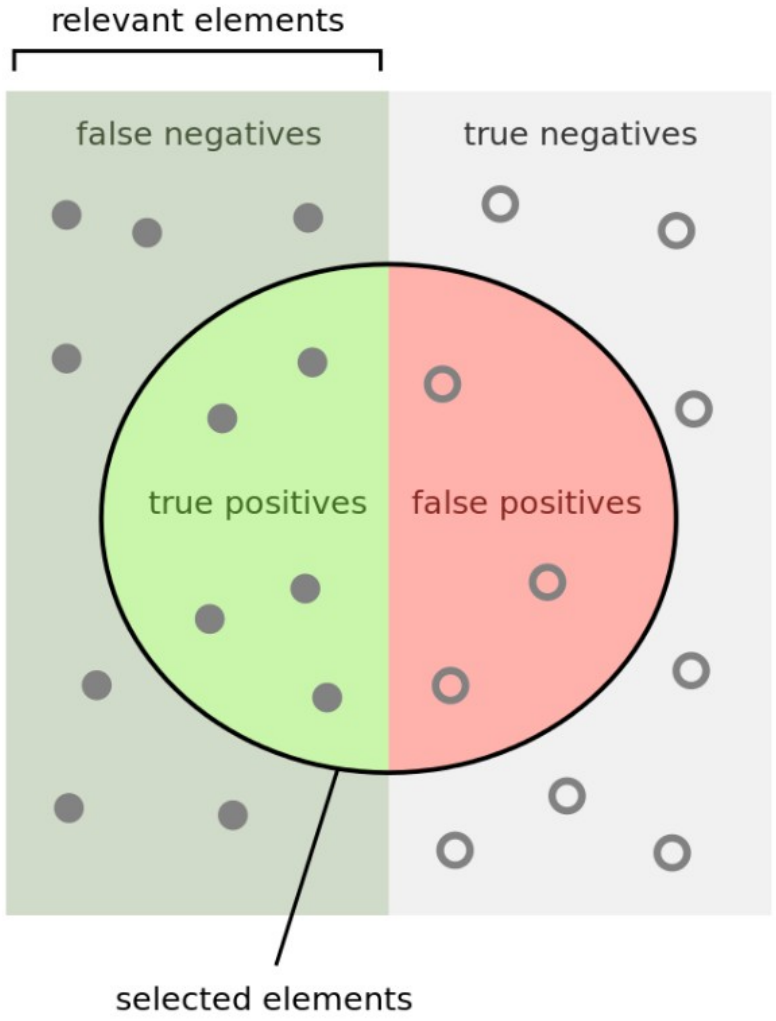
精确率：正类预测的正确程度

Precision = TP / (TP + FP)

召回率：被检出的实际正例比例

Recall = TP / (TP + FN)

两个指标的取值范围均为 0 到 1。



https://en.wikipedia.org/wiki/Precision_and_recall

精确率、召回率与 F 值

- 改变阈值通常在精确率与召回率之间权衡
- ***使用与任务优先级相符的汇总指标***
- **F 度量（或 F 值）：** 精确率与召回率的调和平均
  - 一般地，它是精确率与召回率的加权度量

$$F_\beta = \frac{1}{\alpha \cdot \frac{1}{P} + (1-\alpha)\cdot \frac{1}{R}} = \frac{(\beta^2+1)P * R}{\beta^2 P + R}$$

β > 1 偏重召回率； β < 1 偏重精确率

  - ***F1 度量（平衡 F 度量）***
    - 即当 β = 1 时，

$$F_1 = \frac{2P * R}{P + R}$$

这里 $\alpha = 1/(1+\beta^2)$。 β = 1 时精确率与召回率权重相同。

留出法评估：不要动用测试集

数据子集	用途	决策示例
训练	拟合参数	树的划分或回归权重
验证	选择模型与设置	树深度、 k 、正则化、阈值
测试	最终估计	报告所选模型的性能

- 需要保持类别比例时，对随机划分进行分层。
- 对时序或分组数据，按部署场景进行划分。
- 反复依据测试结果调参，会把测试集变成验证数据。

交叉验证：为模型选择重复利用数据

- 把开发数据分成 K 折（通常 5 或 10）。
- 在 K−1 折上训练，在剩余一折上验证。
- 重复 K 次；汇总验证得分以比较设置。
- 在全部开发数据上重新拟合所选流程，再在测试集上评估。

缩放、特征选择与重采样须在每个训练折内单独拟合。

分层交叉验证保持类别比例；留一法即 K = n。

ROC 曲线：跨阈值的排序

- 改变阈值以描出曲线。
- **纵轴**：真正例率（召回率）。
- **横轴**：假正例率。
- 随机排序的期望 AUC = 0.5；完美分离时 AUC = 1。



ROC AUC 比较的是排序；部署时仍需决策阈值。

精确率–召回率：关注正类

- PR 曲线描绘阈值变化时精确率随召回率的变化。
- 它直接展示找出正例与误报之间的权衡。
- 平均精确率（AP）汇总整个排序上的性能。

示例数据集	始终预测为负	随机排序基线
2% 正例	准确率 98%；召回率 0	期望精确率 ≈ 2%

报告指标时同时给出患病率（正例比例）与工作阈值。

选择与决策相匹配的指标

目标	有用的指标 / 视角	注意
避免漏掉正例	召回率（灵敏度）	检查误报负担
限制误报	精确率	检查漏掉多少正例
平衡精确率与召回率	F_1 or Fβ	不包含真负例
比较排序	ROC AUC； PR / AP	不选择阈值
类别频率不等	各类召回率；平衡准确率	总体准确率可能掩盖失败

选择指标，就是选择哪些错误和权衡更重要。

集成：为什么要组合模型？

- 训练若干基模型，再组合它们的预测。
- 分类：投票或平均类别概率。
- 回归：视方法对预测取平均或相加。
- 当有用模型的错误不完全相同时，组合的收益最大。

两个核心思想： bagging 降低方差；
boosting 顺序地构建模型。

集成方法：提高准确率

- 生成更好的模型需要满足什么条件?
  - 示例：多数投票

基模型性能 →

集成性能 →

	x_1	x_2	x_3
M_1	✓	✓	✗
M_2	✗	✓	✓
M_3	✓	✗	✓
投票集成	✓	✓	✓

情形 1：
集成有正面效果

	x_1	x_2	x_3
M_1	✓	✓	✗
M_2	✓	✓	✗
M_3	✓	✓	✗
投票集成	✓	✓	✗

情形 2：
集成没有效果

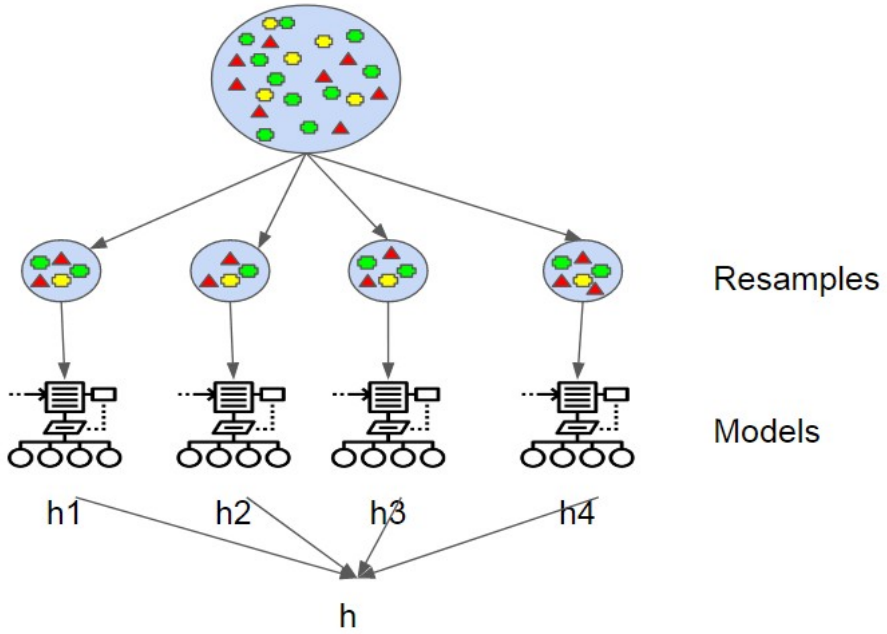
	x_1	x_2	x_3
M_1	✓	✗	✗
M_2	✗	✓	✗
M_3	✗	✗	✓
投票集成	✗	✗	✗

情形 3：
集成有负面效果

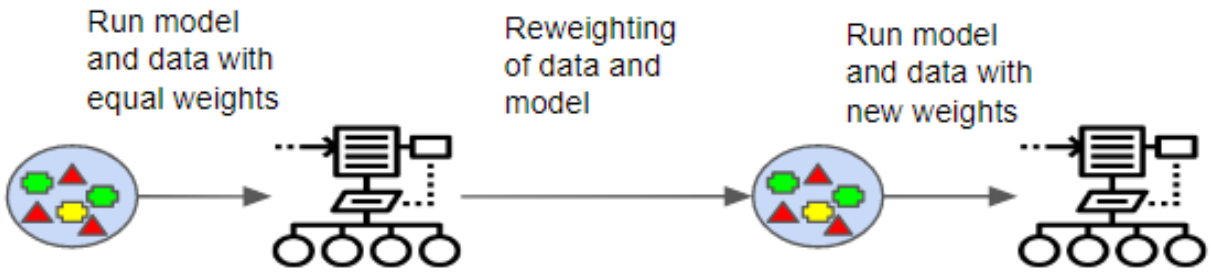
- 基模型应当
  - 准确
  - 多样

集成方法：提高准确率

- 常用集成方法
  - Bagging：在自助样本上训练模型，可并行训练
  - Boosting：顺序训练模型以改进当前集成



Bagging （装袋）



Boosting （提升）

Bagging： 自助聚合

- 类比：根据多位医生的多数意见进行诊断
- 训练
  - For i = 1 to k
    - 对 D 有放回抽样，得到自助样本 D_i；
    - 用 D_i 和学习方案得到模型 M_i；
- 分类：对未知样本 **X** 分类
  - 让 k 个模型分别对 X 分类，返回多数票
- 预测：
  - 预测连续变量时，用平均预测代替投票

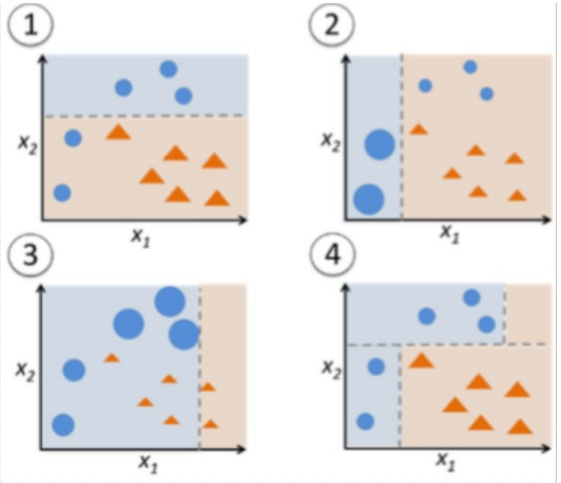
Boosting： AdaBoost 的思想

- 按与加权训练误差相关的权重组合基分类器
- Boosting 如何工作?
  - 迭代地学习一系列 k 个分类器
  - AdaBoost 中，提高被误分类训练样本的相对权重
  - 用加权投票组合基分类器
- Boosting 算法可扩展到数值预测

Adaboost (Freund and Schapire, 1997)

1. 为每个训练元组赋初始权重

3. 根据当前模型更新权重



$\{w_n^{(1)}\}$ $\{w_n^{(2)}\}$ ... $\{w_n^{(k)}\}$

M_1 M_2 ... M_k

2. 在加权数据集上训练基分类器

两种“加权”策略：

1. 为每个训练样本赋权重
2. 按权重分布对数据集抽样

$$M^*(x) = \text{sign}\left(\sum_{i=1}^{k} \alpha_i M_i(x)\right)$$

4. 基分类器训练完后，组合成最终分类器

梯度提升：修正当前模型

- 从简单的初始预测 $F_0(x)$ 开始。
- 用一棵小树拟合当前模型的负损失梯度。
- 加入该树的缩放版本，然后重复。

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

平方误差回归：拟合残差 $y - F_{m-1}(x)$ 。
分类：使用分类损失的梯度。

学习率、树的数量与树深度控制复杂度。

随机森林：基本概念

- 随机森林（L. Breiman 于 2001 年首次提出）
  - 以**决策树**为基模型的 Bagging
  - *数据装袋*
    - 为每棵树抽取一个自助样本（有放回）
  - *特征装袋* ← 随机特征降低树之间的相关性
    - 在每个结点**随机选取部分属性**作为候选，按其中最佳属性划分
  - 分类时每棵树投票，返回得票最多的类别

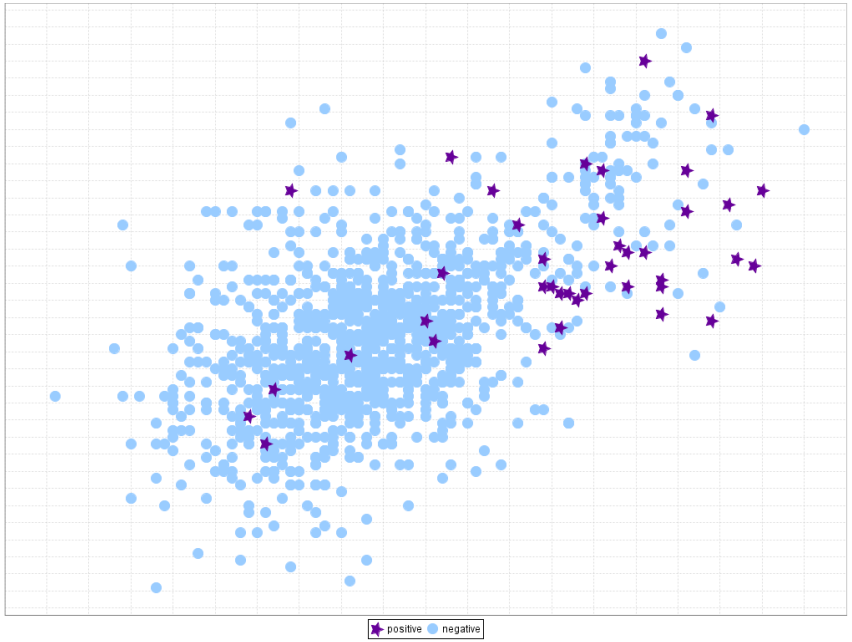
Bagging、随机森林与 Boosting

方法	基模型如何不同	组合方式
Bagging （装袋）	训练数据的自助样本	投票 / 平均
随机森林	自助样本 + 随机特征子集	投票 / 平均概率
AdaBoost	重新加权的训练样本	加权投票
梯度提升	依次拟合负损失梯度	累加缩放后的模型输出

验证集成规模、基模型复杂度及任务相关性能。

当类别不平衡时

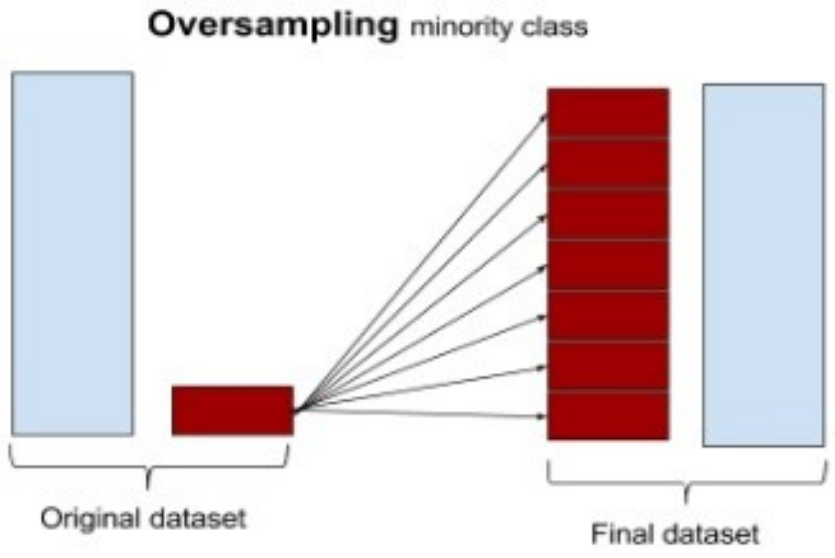
- 稀有类别可能恰恰是我们最需要检测的类别。
- 例如：欺诈、故障和筛查阳性结果。
- 总体准确率可能掩盖少数类的糟糕表现。
- 根据应用选择错误代价与评估指标。

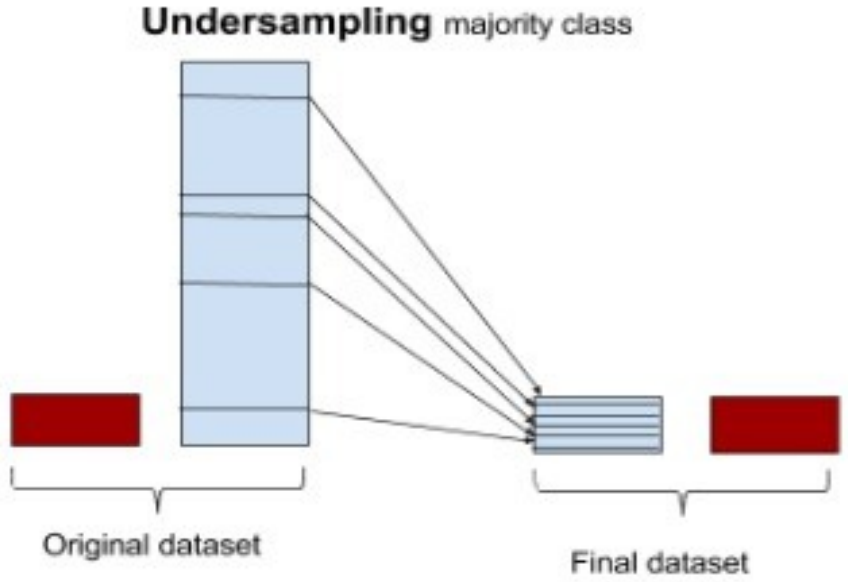


类别不平衡与错误代价不等相关，但是不同的问题。

类别不平衡数据集的分类

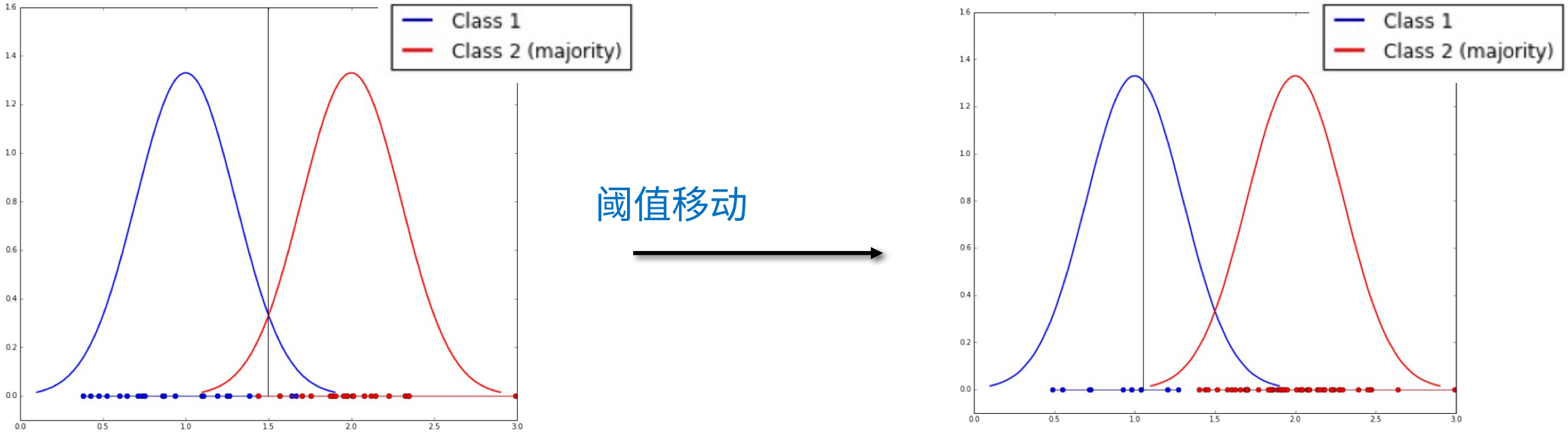
- 数据层面方法：只修改训练数据
  - **过采样**：对少数类过采样。
  - **欠采样**：随机删除多数类元组
  - **合成采样：生成新的少数类样本**





类别不平衡数据集的分类

- 算法 / 决策层面方法
  - **阈值移动：降低 t 以预测更多正例；召回率和假正例通常都会增加**
  - **类别权重：对任务中重要的类别或错误赋予更大权重**
- **集成：结合采样或加权策略组合模型**



很小的假正例率也可能很重要

10,000 个样本： 200 个正例， 9,800 个负例。
在某一阈值下：召回率 = 80% ，假正例率 = 1% 。

量	计算	结果
真正例	0.80 × 200	160
假正例	0.01 × 9,800	98
精确率	160 / (160 + 98)	≈ 62%

解读比率时要结合正例比例与实际错误数。

不平衡数据的可靠流程

- 定义正类以及漏报与误报的代价。
- 做无泄漏的划分；让测试集的正例比例符合实际。
- 只在训练中使用重采样或类别权重。
- 在验证数据上调整决策阈值。
- 报告混淆计数、精确率和召回率；必要时补充排序指标。

与简单基线比较，而不只看准确率。

快速检查： 98% 的准确率好吗？

问题：测试集有 200 个正例和 9,800 个负例。某分类器把所有样本都预测为负。

	预测为正	预测为负
实际为正	0	200
实际为负	0	9,800

- 其召回率是多少？ 98% 的准确率能说明它有用吗？
- 你还会报告什么指标或决策准则？

分类：完整流程

步骤	主要要点
1. 定义与准备	选择目标；在需学习的预处理之前划分数据
2. 拟合模型	让模型假设与容量与数据相匹配
3. 验证选择	在不泄漏测试数据的前提下调节设置、正则化与阈值
4. 评估	使用与正类及错误代价一致的指标
5. 改进	使用集成或不平衡处理；验证其收益

强分类器既需要可靠的模型，也需要可信的评估。

第 2 次课附录

附录 2：最小二乘解

单个输入特征时： $\hat{y}_i = wx_i + b$

最小化 $L(w,b) = \Sigma_i (y_i - wx_i - b)^2$

$$w = [\sum_i (x_i - \bar{x})(y_i - \bar{y})] / [\sum_i (x_i - \bar{x})^2]$$

$$b = \bar{y} - w\bar{x}$$

- $\bar{x}$ 与 $\bar{y}$ 为样本均值。
- 分母必须非零： x 在各样本间必须有变化。

附录 2：推导逻辑回归梯度

$$\ell_i = -y_i \ln(p_i) - (1 - y_i) \ln(1 - p_i)$$

$$p_i = \sigma(z_i), \qquad z_i = w^T x_i + b$$

$$\partial \ell_i / \partial z_i = p_i - y_i$$

$$\nabla w \, L = (1/n) \sum_i (p_i - y_i) x_i$$

$$\partial L / \partial b = (1/n) \sum_i (p_i - y_i)$$

梯度下降减去这些梯度；对数似然上升则加上其梯度。

快速检查：读懂混淆矩阵

- 用同一个混淆矩阵，计算刚介绍的指标

实际类别 \ 预测类别	cancer = yes	cancer = no	合计
cancer = yes	**90**	**210**	300
cancer = no	**140**	**9560**	9700
合计	230	9770	10000

  - Sensitivity =
  - Specificity =
  - Accuracy =
  - Error rate =
  - Precision =
  - Recall =
  - F1 =

快速检查：答案与解读

- 用同一个混淆矩阵，计算刚介绍的指标

实际类别 \ 预测类别	cancer = yes	cancer = no	合计
cancer = yes	**90**	**210**	300
cancer = no	**140**	**9560**	9700
合计	230	9770	10000

- Sensitivity = TP/P = 90/300 = 30%
- Specificity = TN/N = 9560/9700 = 98.56%
- Accuracy = (TP + TN)/All = (90+9560)/10000 = 96.50%
- Error rate = (FP + FN)/All = (140 + 210)/10000 = 3.50%
- Precision = TP/(TP + FP) = 90/(90 + 140) = 90/230 = 39.13%
- Recall = TP/ (TP + FN) = 90/(90 + 210) = 90/300 = 30.00%
- F1 = 2 P × R /(P + R) = 2 × 39.13% × 30.00%/(39.13% + 30%) = 33.96%

附录 2：解读一次逻辑回归更新

$$w \leftarrow w + \eta(y - p)x \qquad b \leftarrow b + \eta(y - p)$$

真实类别	误差信号 $y - p$	对该样本的影响
$y = 1$	$1 - p > 0$	提高其线性得分与概率
$y = 0$	$-p < 0$	降低其线性得分与概率

- 自信的错误预测会产生更大的修正。
- 接近正确的概率只产生较小的修正。

附录 2：二分类 AdaBoost 细节

标签与基预测：$y_i, h_m(x_i) \in \{-1, +1\}$。初始 $D_1(i) = 1/n$。

$$\varepsilon_m = \sum_i D_m(i)\, I[y_i \neq h_m(x_i)]$$

$$\alpha_m = \tfrac{1}{2} \ln[(1 - \varepsilon_m) / \varepsilon_m]$$

$$D_{m+1}(i) \propto D_m(i) \exp[-\alpha_m\, y_i\, h_m(x_i)]$$

$$\text{最终预测} = \operatorname{sign}[\textstyle\sum_m \alpha_m\, h_m(x)]$$

要求学习器优于随机猜测（$\varepsilon_m < 0.5$）；$\varepsilon_m = 0$ 时停止。

附录 2： 随机森林细节

- 随机输入选择（RF-RI)： 每次划分时尝试一个随机特征子集。
- 随机组合变体（RF-RC)： 用特征的随机组合进行划分。
- 袋外样本未进入某棵树的自助样本，可用于估计该树集成的性能。
- 调节特征子集大小、叶大小 / 深度与树的数量；检查各类别的错误。

随机性带来多样性，但不能保证不会过拟合。